

Week 2 – Data Structures and Functions

Objective

The objective of Week 2 was to understand Python data structures and functions and how they can be used to store, process, and transform data. This week focused on learning lists, tuples, dictionaries, sets, functions, lambda functions, recursion, and list comprehension.

1. Lists

Lists are ordered collections that can store multiple values in a single variable. Lists are mutable, meaning their elements can be modified after creation.

Example:

```
numbers = [10, 20, 30, 40]
```

2. Tuples

Tuples are similar to lists but they are immutable. Once created, their elements cannot be changed.

Example:

```
coordinates = (10, 20)
```

3. Dictionaries

Dictionaries store data as key–value pairs. Each key is associated with a value.

Example:

```
student = {"name": "abc", "age": 20}
```

4. Sets

Sets are unordered collections that store only unique elements. Duplicate values are automatically removed.

Example:

```
numbers = {1, 2, 3, 3, 4}  
# Output: {1, 2, 3, 4}
```

5. Functions

Functions are reusable blocks of code that perform specific tasks.

Example:

```
def add(a, b):  
    return a + b
```

6. Lambda Functions

Lambda functions are small anonymous functions used for short operations.

Example:

```
square = lambda x: x * x
```

7. Recursion

Recursion is a method where a function calls itself.

Example:

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n-1)
```

8. List Comprehension

List comprehension provides a concise way to create lists.

Example:

```
squares = [x*x for x in range(5)]  
# Output: [0, 1, 4, 9, 16]
```

Program Implemented – 1: Data Cleaning Script

Description

A Python script was developed to perform data cleaning operations such as removing duplicate values, filtering even numbers, and calculating the sum of squares.

Program Code

```
# List containing duplicate values  
numbers = [10, 20, 20, 30, 40, 40, 50, 60]  
  
print("Original Data:", numbers)  
  
# Function to remove duplicates  
def remove_duplicates(data):
```

```

    return list(set(data))

# Lambda function to filter even numbers
filter_even = lambda data: [x for x in data if x % 2 == 0]

# Function to calculate sum of squares
def sum_of_squares(data):
    return sum([x**2 for x in data])

# Remove duplicates
clean_data = remove_duplicates(numbers)
print("After Removing Duplicates:", clean_data)

# Filter even numbers
even_numbers = filter_even(clean_data)
print("Filtered Even Numbers:", even_numbers)

# Sum of squares
result = sum_of_squares(even_numbers)
print("Sum of Squares:", result)

```

OUTPUT:

```

PS C:\Users\HP\Documents\Data Science Month-1 ( D. Pravallika)\Week - 2> python Data_cleaning.py
Original Data: [10, 20, 20, 30, 40, 40, 50, 60]
After Removing Duplicates: [40, 10, 50, 20, 60, 30]
Filtered Even Numbers: [40, 10, 50, 20, 60, 30]
Sum of Squares: 9100

```

Program Implemented – 2: Remove Duplicates

Description

This program removes duplicate elements from a list using sets.

Program Code

```

data = list(map(int, input("Enter numbers: ").split()))
unique_data = list(set(data))
print("After removing duplicates:", unique_data)

```

OUTPUT:

```

PS C:\Users\HP\Documents\Data Science Month-1 ( D. Pravallika)\Week - 2> python Remove_Duplicates.py
Enter numbers: 7 9 4 7 8 9
After removing duplicates: [8, 9, 4, 7]

```

Program Implemented – 3: Filter Data

Description

This program filters numbers greater than a given value.

Program Code

```
data = list(map(int, input("Enter numbers: ").split()))
threshold = int(input("Enter filter value: "))
```

```
filtered = [x for x in data if x > threshold]
print("Filtered Data:", filtered)
```

OUTPUT:

```
PS C:\Users\HP\Documents\Data Science Month-1 ( D. Pravallika)\Week - 2> python Filter_Data.py
Enter numbers: 7 4 9 6 55
Enter filter value: 6
Filtered Data: [7, 9, 55]
```

Program Implemented – 4: Sum of Squares

Description

This program calculates the squares of numbers using list comprehension.

Program Code

```
data = list(map(int, input("Enter numbers: ").split()))
squares = [x**2 for x in data]
print("Squares:", squares)
```

OUTPUT:

```
PS C:\Users\HP\Documents\Data Science Month-1 ( D. Pravallika)\Week - 2> python Sum_of_squares.py
Enter numbers: 7
Squares: [49]
```

Conclusion

Week 2 enhanced understanding of Python data structures and functions. It also demonstrated how to clean, filter, and transform data efficiently using Python.